



به نام خدا

دانشکده‌ی مهندسی برق و کامپیوتر

یادگیری تعاملی - پاییز ۱۴۰۳ – ۱۴۰۴

تمرین سری سوم

فاطمه محمدی – ۸۱۰۱۹۹۴۸۹

سوال اول: ۲

تاثیر پاداش تنک، در نرخ همگرایی: ۲

روشهای پیشنهادی برای حل مشکلات ناشی از تنک بودن پاداش: ۲

مشکلات دیگر مربوط به مسائل انسان در حلقه، به همراه راه‌حلهای پیشنهادی: ۳

سوال دوم: ۵

مقایسه از لحاظ بروزرسانی مقدار Q: ۵

مقایسه پایداری، سرعت همگرایی: ۶

شرایط مناسب استفاده: ۶

سوال سوم: ۷

تفاوت الگوریتمهای on-policy و off-policy از منظر بهره‌وری نمونه، اکتشاف محیط و مقاومت در برابر تصادفی بودن محیط: ۷

در مسائل دنیای واقعی کدام الگوریتم عملکرد بهتری دارد؟ ۷

سوال چهارم: ۹

بررسی محیط: ۹

الف) ۱۰

ب) ۱۱

ج) ۱۲

منابع: ۱۳

سوال اول: ۱۳

سوال دوم: ۱۳

سوال سوم: ۱۳

سوال چهارم: ۱۳

سوال اول:

تاثیر پاداش تنک، در نرخ همگرایی:

پاداشهای تنک، تأثیر قابل توجهی بر نرخ همگرایی الگوریتم Q-learning (و سایر الگوریتم‌های یادگیری تقویتی) دارند.

۱. Delayed Reward

پاداش تنک، به این معناست که عامل به ندرت و دیر به دیر، پاداش (بازخورد) دریافت میکند و این مسئله به وضوح میتواند باعث شود عامل دیرتر محیط را بشناسد (دیر به دیر عمل "اپدیت کردن" را انجام دهد) و در نتیجه دیرتر به پالیسی (حتی غیر بهینه) همگرا شود.

۲. Impact on Q-value Updates:

همانطور که قبلتر اشاره کردیم، پاداش تنک باعث میشود مقادیر Q دیرتر و کمتر اپدیت شوند، همین میتواند موجب کاهش نرخ همگرایی شود.

۳. Exploration Challenges and Potential for Exploration Bias:

پاداشهای تنک نیازمند اکتشاف بیشتر هستند تا عامل بتواند حالتها (states) یا عملها (actions) معناداری موجب دریافت پاداش (بازخورد) میشوند را پیدا کند. درصورتی استراتژی اکتشاف عامل کارامد نباشد (مثلا عامل به اندازه کافی رفتار تصادفی نشان ندهد)، یادگیری کندتر خواهد شد.

پاداشهای تنک میتوانند در تخمین مقادیر Q، بایاس ایجاد کنند، اگر عامل نتواند به اندازه کافی محیط را کاوش کند. این بایاس به دلیل رویت نکردن همه حالتها و عملهای شامل پاداش ممکن، ایجاد میشود.

۴. Credit Assignment Problem:

در پاداشهای تنک، تخصیص اعتبار به عملهایی که منجر به پاداش میشوند دشوارتر است (عامل باید بتواند اثر عملها را در چندین مرحله به عقب ردگیری کند که این فرایند را پیچیده تر (درنتیجه کندتر) میکند.

البته در تمامی موارد بالا فرض شده، به پالیسی (سیاست) بهینه میرسیم، درواقع پاداشهای تنک میتواند موجب افزایش نرخ همگرایی شوند اما این معمولا به قیمت همگرایی به پایسی غیربیهنه تمام میشود، که در ادامه لیست خلاصه ای از دلایل این مورد را اورده ایم:

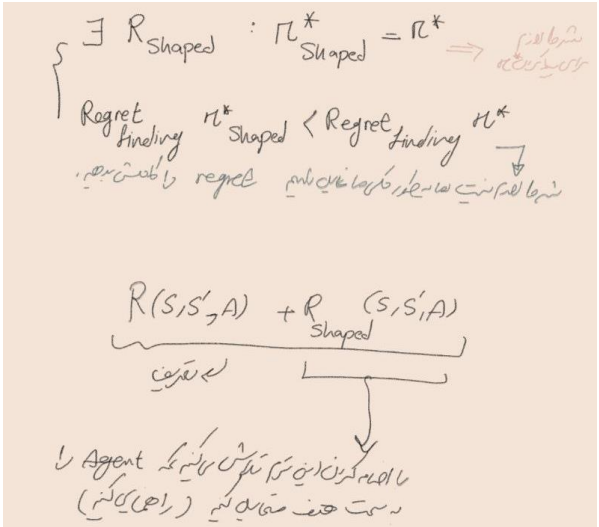
۱. Early Overestimation or Greedy Behavior
۲. Simpler Feedback for Learning
۳. Narrow Exploration

روشهای پیشنهادی برای حل مشکلات ناشی از تنک بودن پاداش:

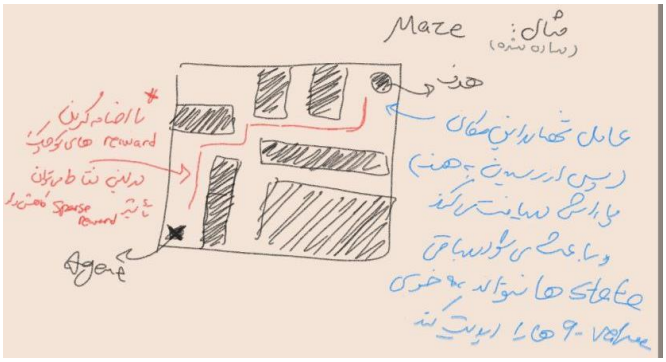
۱. Reward Shaping (در کلاس بحث شده)

درواقع با تغییر شکل reward ها (به صورت نشان داده شده در تصویر زیر)، تلاش میکنیم عامل را به سمت هدف هدایت کنیم.

- البته لازم است دو شرط رعایت شود، که در شکل زیر نشان داده ایم:



- مثال:



۲. Curriculum Learning و Hierarchical Reinforcement Learning

معمولا در دنیای واقعی برای یادگیری به تدریجی مسائل و مطالب سخت تر یاد داده میشوند (برای مثال اول به یادگیری جمع بعد ضرب و بعد توان پرداخته میشود) اما برای عامل ها در RL به این صورت نمیباشد، اما این روش نیز میتواند در صورت امکان یه راه حل مناسب برای مسائل انسان در حلقه باشد.

در این حالت، ابتدا یادگیری با وظایف ساده تر و پاداشهای متراکم تر شروع میشود و به تدریج به وظایف پیچیده تر با پاداشهای پراکنده منتقل میشود.

- مثال:

آموزش عامل برای یادگیری مهارتهای ابتدایی مسیریابی پیش از معرفی Maze پیچیده.

تقسیم یک وظیفه پیچیده به وظایف کوچکتر و سپس ترکیب انها.

۳. Social Learning

در صورت امکان استفاده از روشهای یادگیری اجتماعی میتواند بسیار موثر باشد و د این راه حل میتوانیم از تکنیکهای متنوعی استفاده کرد:

- مشاهده: یادگیری از طریق مشاهده اعمال و نتایج دیگر عوامل. (معمولا به دلیل privacy به نتایج دسترسی نداریم، اما با توجه به مشاهده تنها همان عملهای پرتکرار دیگر عوامل نیز میتوان نرخ همگرایی را بهبود داد.)
- همکاری: به اشتراک‌گذاری سیاستها، مقادیر Q یا تجربیات در یک سیستم چندعاملی.
- بازخورد از دیگران: استفاده از پاداشها یا جریمه هایی که توسط عاملهایی یا هدف یکسان یا انسان ها ارائه میشود.

- نکته قابل توجه در این روشها این هست باید یک تعادل بین بازخوردهای واقعی خود و بازخوردهای دریافتی از دیگر عوامل و یا مشاهدات ایجاد کرد چرا که یک مسئله مهم در مسائل انسان در حلقه diversity افراد میباشد و باید به بازخوردهای واقعی به طور معمول ارزش بیشتری داد.)

۴. Monte Carlo Methods

جمع‌آوری پاداشهایی که کل اپیزود گرفته ایم میتواند موجب شود ارزش عملها و یا حالتها را برای کل مسیر تخمین زد.

۵. Eligibility Traces

استفاده از Eligibility Traces برای انتشار پاداشهای تنک (پراکند) در کل مسیر (ترکیب روشهای Monto Carlo با Temporal (Difference (TD) learning

۶. Exploration Strategies and Incentivize Exploration

۷. Imitation Learning

مشکلات دیگر مربوط به مسائل انسان در حلقه، به همراه راه‌حلهای پیشنهادی:

۱. Sparse Rewards و Delayed Feedback

به این مورد قبلتر پرداختیم.

۲. Non-Stationary

رفتار، اهداف و حتی ترجیحات افراد ممکن است در طول زمان تغییر کنند. این موضوع باعث میشود که محیط ناپایدار شود و پاداش یا خروجی های مطلوب فرد در حین یادگیری عامل تغییر کنند.

- راه حل:

استفاده از مدلهای پویا که با بازخورد افراد منطبق و بروز رسانی میشوند.
پیاده سازی الگوریتم هایی برای تشخیص و پاسخ به تغیرات در الگوهای پاداش.

۳. Ambiguity in Feedback

بازخورد انسانی ممکن است نامشخص یا ناهماهنگ باشد.

- راه حل:

در نظر گرفتن بازخورد افراد به‌عنوان سیگنالی احتمالاتی و نه قطعی (و استفاده از تکنینکهایی مانند بیشینه سازی احتمال)
یادگیری فعال: طراحی سیستمی که در صورت مبهم بودن بازخورد افراد، از انان سوالات واضح بپرسد.
ترکیب چند نوع مختلف بازخورد.
....

۴. Human Error

رفتار و بازخورد افراد میتواند به هر علتی نادرست باشد.

- راه حل:

فیلتر کردن نویز: استفاده از الگوریتم هایی برای شناسایی نقاط پرت و حذف بازخوردهای اشتباه.
یادگیری اجتماعی: جمع اوری بازخورد از چند نفر و میانگین گیری (البته این روش همواره راه حل مناسبی نمیباشد بخصوص اگر هدف شخصی سازی برای یک فرد خاص باشد.)

۵. Diverse Goals

افراد ممکن است اهداف متناقض یا متنوعی داشته باشند که باعث ناهماهنگی در بازخورد شود.

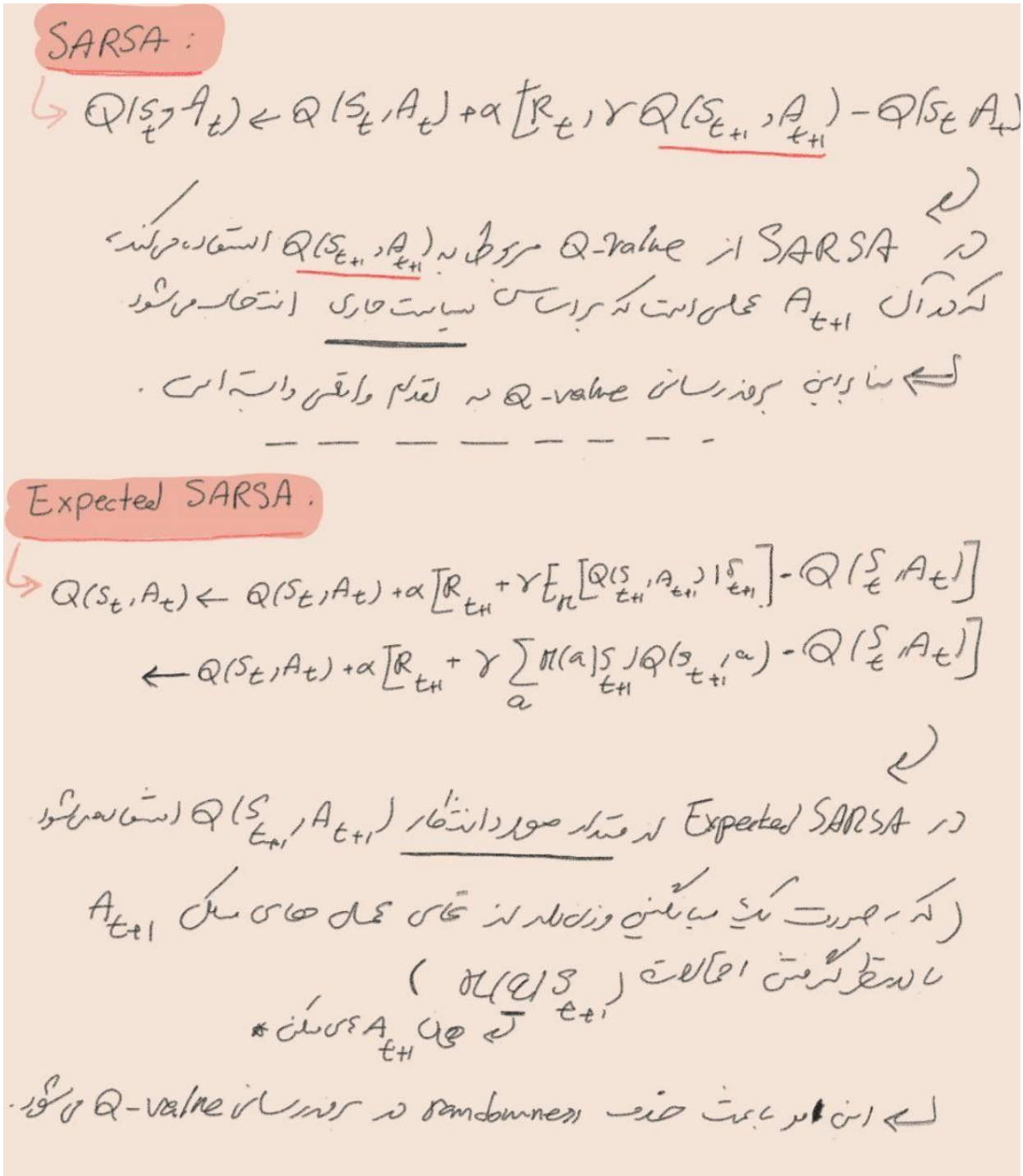
- راه حل:

این مشکل خصوصا در زمانی که از روشهای یادگیری اجتماعی استفاده میکنیم میتواند مشکل ساز باشد و در این موارد میتوان با برقراری تعادل و تلاش برای ارزش گذاری بیشتر برای بازخوردهای واقعی میتوانیم از این مشکل تاحدی جلوگیری کنیم و در عین یادگیری اجتماعی هم داشته باشیم.

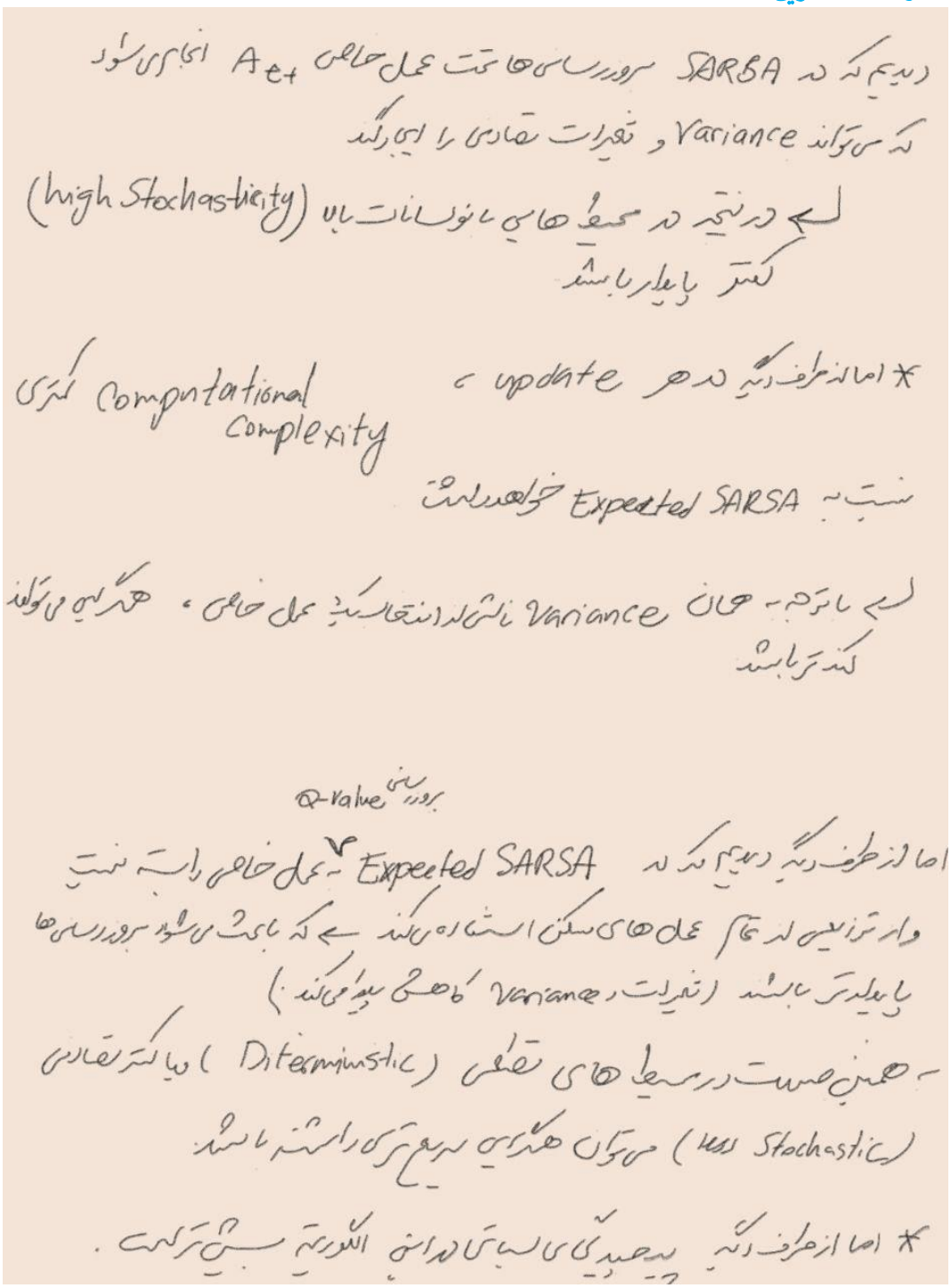
۶. ...

سوال دوم:

مقایسه از لحاظ بروزرسانی مقدار Q:



مقایسه پایداری، سرعت همگرایی:



دیدیم که در SARSA بروزرسانی‌ها تحت عمل خاص A_{t+1} انجام می‌شود
که می‌تواند Variance و تغییرات تصادفی را ایجاد کند
← در نتیجه در محیط‌های با نوسانات بالا (High Stochasticity)
کمتر پایدار باشد

* اما از طرف دیگر در هر update ، کمتری Computational Complexity

نسبت به Expected SARSA خواهد داشت

← با توجه به Variance بالاتر در انتخاب عمل خاص ، همگرایی می‌تواند
کندتر باشد

بروزرسانی Q-value

اما از طرف دیگر دیدیم که در Expected SARSA ← عمل خاص را نسبت
وارد ترازیابی در تمام عمل‌های ممکن استفاده می‌کند ← که باعث می‌شود بروزرسانی‌ها
پایدارتر باشد (تغییرات Variance کاهش پیدا می‌کند.)
- همین حساسیت در محیط‌های قطعی (Deterministic) یا کمتر تصادفی
(Less Stochastic) می‌توان همگرایی سریع‌تری داشته باشد

* اما از طرف دیگر پیچیدگی‌ی بیشتری در این الگوریتم بیشتر است .

شرایط مناسب استفاده:

SARSA	Expected SARSA
• محیط‌های پویا و یا تصادفی ← در آن‌ها نمونه‌برداری از یک عمل A_{t+1} می‌تواند نماینده خوبی از عمل‌های موجود باشد	• محیط‌های قطعی یا کمتر تصادفی ← که در آن‌ها می‌توان به صورت موثرتر توزیع عمل‌ها را استفاده کرد
• مناسب زمانی‌که منابع محاسباتی محدود است.	• موقعیت‌هایی‌که سرعت همگرایی و یا پایداری اهمیت زیادی دارند

سوال سوم:

تفاوت الگوریتمهای on-policy و off-policy از منظر بهره‌وری نمونه، اکتشاف محیط و مقاومت در برابر تصادفی بودن محیط:

۱. **بهره‌وری نمونه:**

الگوریتمهای On-Policy:

در این الگوریتمها فقط از داده هایی استفاده میشود که تحت سیاست فعلی تولید شده اند. این موضوع باعث میشود بهره‌وری نمونه کمتری داشته باشند، زیرا برای بهبود سیاست نیاز به تعاملات بیشتری با محیط دارند.

برای مثال: الگوریتم SARSA از اقدامات واقعی انجام شده توسط سیاست فعلی برای یادگیری استفاده میکند که محدود شده به داده‌های حاصل از تجربیات سیاست فعلی ختم است.

الگوریتمهای Off-Policy:

در این الگوریتمها میتوانیم از داده هایی که توسط سیاستهای دیگر یا داده هایی تولید شده قدیمی استفاده کنند. این قابلیت باعث میشود بهره‌وری نمونه بیشتری داشته باشند.

برای مثال: الگوریتم Q-learning از اکشنهای بهینه طبق Q-value یادگرفته شده استفاده میکند، حتی اگر این اکشنها در هنگام کاوش انجام نشده باشند.

۲. **کاوش:**

الگوریتمهای On-Policy:

این الگوریتمها کاوش و بهره‌برداری را به صورت مستقیم در چارچوب سیاستی که دنبال میشود، متعادل میکنند. برای مثال، SARSA به استراتژی کاوش (مثلا ε-greedy) در طول به‌روزرسانی ها پایبند است.

- مزیت: تضمین ایمنی در وظایفی که اقدامات باید محدود به سیاست فعلی باشند.

الگوریتمهای Off-Policy:

کاوش مستقل از سیاست یادگیری انجام میشود که به الگوریتمها اجازه میدهد استراتژی های کاوش قوی تری را به کار گیرند.

این میتواند منجر به کشف سیاستهای بهتر شود.

- مزیت: مناسب برای محیط هایی که کاوش اهمیت زیادی دارد.

۳. **مقاومت دربرابر تصادفی بودن محیط**

الگوریتمهای On-Policy:

در محیطهای تصادفی مقاومت بیشتری دارند، زیرا به‌صورت ذاتیع ملهای انجام شده تحت سیاست فعلی و نتایج انها را در نظر میگیرند. به عبارتی بروزرسانی ها بازتابی از بازدهی واقعی سیاست فعلی هستند.

برای مثال: SARSA پاداش و حالت ناشی از اقدام اکتشافی را در نظر میگیرد، که آن را برای محیط‌های پر نویز و یا تصادفی مناسبتر میکند.

الگوریتمهای Off-Policy:

در محیط های تصادفی ممکن است به دلیل اختلاف بین سیاست اجرایی (برای تولید داده) و سیاست هدف (برای بروزرسانی) دچار تناقض شوند و این موضوع میتواند منجر به ناپایداری شود.

در مسائل دنیای واقعی کدام الگوریتم عملکرد بهتری دارد؟

در مسائل دنیای واقعی، الگوریتمهای Off-Policy معمولا در شرایطی که بهره‌وری نمونه و انعطاف پذیری در کاوش حیاتی است، مانند " مدیریت وسایل نقلیه خودران "، عملکرد بهتری دارند. این الگوریتمها میتوانند از داده های شبیه سازی شده یا قدیمی در مقیاس بزرگ برای بهبود سریع سیاستها استفاده کنند.

با این حال، در محیط های بسیار تصادفی که ایمنی یا همراستایی سیاستها اهمیت زیادی دارد (مانند رباتیک در حوزه پزشکی)، الگوریتمهای On-Policy به دلیل قابلیت اطمینان و پایداری بیشتر ترجیح داده میشوند.

نگاهی دقیقتر به مدیریت وسایل نقلیه خودران (Autonomous Vehicle Management)

صورت مسئله: یک خودرو خودران باید در محیطهای پیچیده و پویا تصمیم گیری کند.

چالش اول: محیط تصادفی است. (مانند رفتار عابران پیاده و وسایل نقلیه دیگر).

چالش دوم: کاوش و یادگیری بدون ایجاد خطر برای مسافران و دیگران.

تحلیل الگوریتم ها:

- On-Policy:

الگوریتمهای On-Policy مانند SARSA زمانی مفید هستند که محدودیتهای ایمنی اهمیت بالایی دارند، زیرا این الگوریتمها سیاست را بر اساس رفتار فعلی به صورت محافظه کارانه بروزرسانی میکنند. برای مثال، SARSA اطمینان میدهد که یک اقدام اکتشافی اما خطرناک (مانند سبقت گرفتن) منجر به تقویت بیش از حد نتایج خطرناک نمیشود.

- Off-Policy:

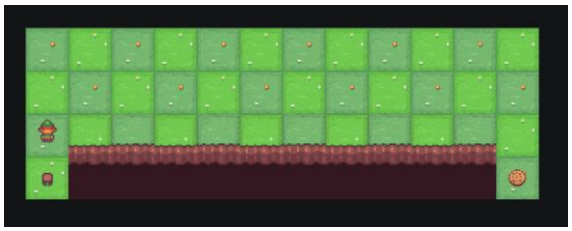
الگوریتمهای Off-Policy، مانند Q-learning یا DQN، میتوانند از داده های شبیه سازی شده یا رفتار رانندگان واقعی استفاده کنند تا سیاست بهینه را یاد بگیرند. این الگوریتم‌ها بهره‌وری نمونه بالایی دارند و نیازی به کاوش های خطرناک در محیط واقعی ندارند.

Off-Policy برای طراحی و توسعه سیاست خودروهای خودران ایده آل است. اما در زمان اجرای سیاستها، برای اطمینان از ایمنی، ترکیب با آن با الگوریتمهای On-Policy میتواند مفید تر باشد.

سوال چهارم:

بررسی محیط:

محیط بازی:



Action Space	`Discrete(4)`
Observation Space	`Discrete(48)`

Action Space

The action shape is `(1,)` in the range `{0, 3}` indicating which direction to move the player.

- 0: Move up
- 1: Move right
- 2: Move down
- 3: Move left

Observation Space ¶

There are 3 x 12 + 1 possible states. The player cannot be at the cliff, nor at the goal as the latter results in the end of the episode. What remains are all the positions of the first 3 rows plus the bottom-left cell.

The observation is a value representing the player's current position as current_row * ncols + current_col (where both the row and col start at 0).

For example, the starting position can be calculated as follows: 3 * 12 + 0 = 36.

The observation is returned as an `int()`.

Starting State

The episode starts with the player in state `[36]` (location [3, 0]).

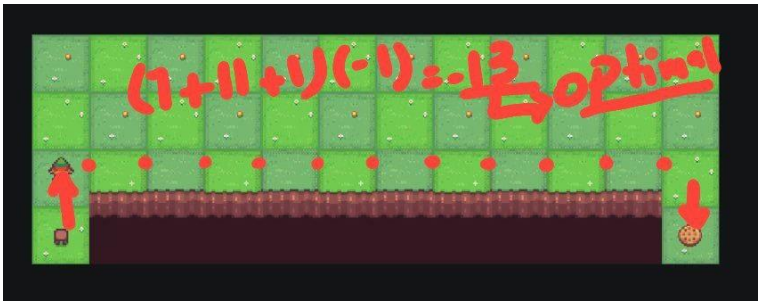
Reward

Each time step incurs -1 reward, unless the player stepped into the cliff, which incurs -100 reward.

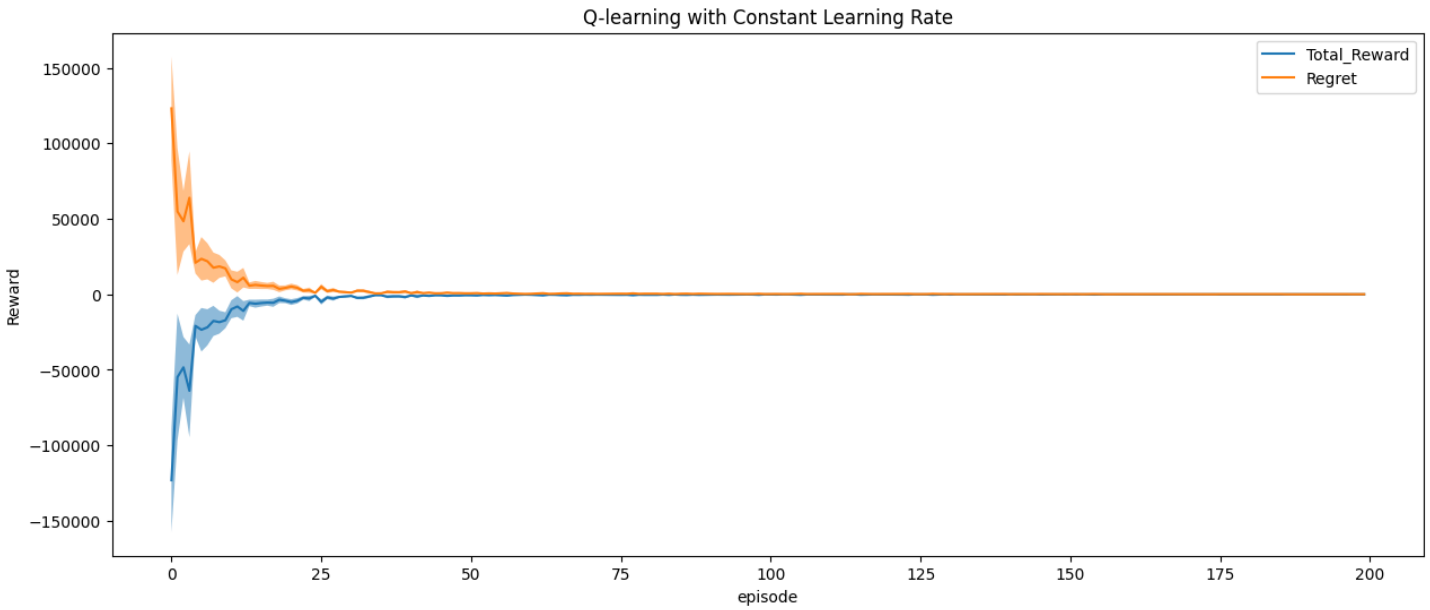
Episode End

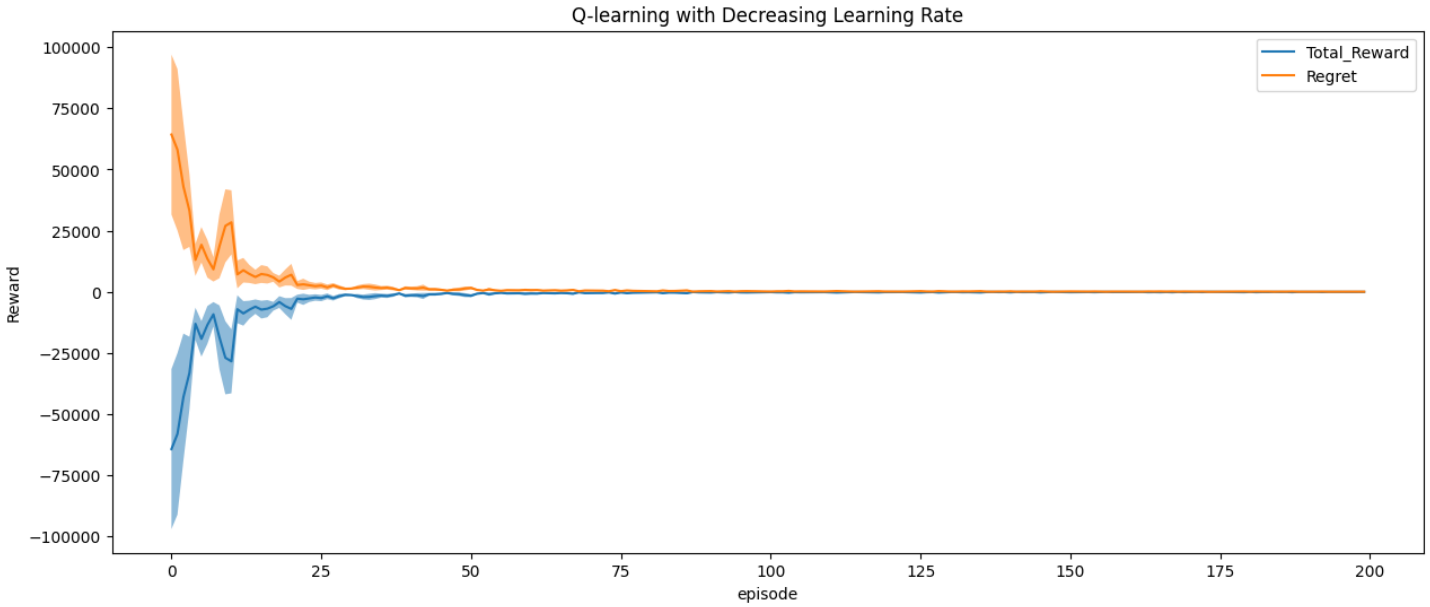
The episode terminates when the player enters state `[47]` (location [3, 11]).

سیاست بهینه: (حداقل ۱۳ حرکت)



الف)





سرعت همگرایی: با توجه به range نمودار های میتواند دید عامل با نرخ یادگیری کاهشی سریعتر همگرا شده است.

عامل با نرخ یادگیری کاهشی پاداشهای کلی بهتری به دست آورد که نشان دهنده یادگیری سریعتر و سازگاری سریعتر با سیاست بهینه است.

اما در نهایت هردو عامل به مقدار -۱۳ طبق انتظار همگرا شده اند.

```
max(max(total_rewards_dec_lr))
✓ 0.0s
-13
```

```
max(max(total_rewards_const_lr))
✓ 0.0s
-13
```

مقدار اپسیلون در طول فرایند اموزش به تدریج کاهش داده ایم تا تعادل مناسبی بین اکتشاف (exploration) و بهره‌برداری (exploitation) برقرار شود.

در ابتدای یادگیری ، اپسیلون مقدار بالایی (۱) دارد تا بیشتر اکتشاف کند چرا که محیط در ابتدا ناشناخته است اما با پیشرفت یادگیری، اپسیلون با استفاده از یک ضریب (۰/۹۹ که یک ضریب رایج است) اپسیلون را به تدریج کاهش میدهیم. این کاهش باعث میشود که در مراحل اولیه یادگیری ، اکتشاف بیشتر انجام شود و در مراحل بعدی، بهره‌برداری از سیاست یادگرفته شده انجام شود. این روند ادامه میابد تا اپسیلون به یک حداقل مقداری (۰/۰۱) برسد که حتی در مراحل پایانی یادگیری هم امکان اکتشاف جزئی وجود داشته باشد.

ب)

*** نمودارها در فایل نوتبوک قرار دارند.

همانطور انتظار میرفت SARSA به عنوان یک روش on-policy در به‌روزرسانی های خود محتاط است، یادگیری آهسته تری دارد که باعث میشود در محیط هایی که شامل جریمه های سنگین هستند، عملکرد ایمنتری ارائه دهد.

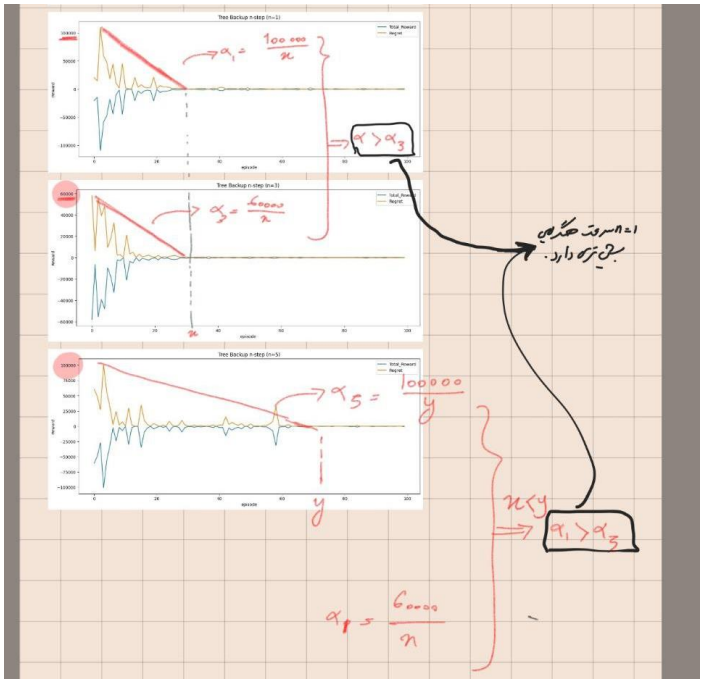
پس SARSA سرعت همگرایی پایینتری دارد و حسرت کلی بیشتری نسبت به روشهای Tree Backup نشان داد که این موضوع با ماهیت on-policy آن مطابقت دارد که به سیاست فعلی وابسته است و در صورت غیر بهینه بودن، باعث یادگیری کندتر و کمتر مؤثر میشود.

نتایج تأیید میکنند که SARSA در محیط هایی مانند Cliff Walking که جریمه های سنگین برای رفتارهای غیر بهینه دارد و یادگیری و گذر زمان هزینه دارد، عملکرد ضعیفی دارد.

همانطور انتظار میرفت مقدار کمتر n روی پاداشهای فوری تاکید دارند و منجر به یادگیری سریعتر اما با اگاهی کمتر میشوند. این روش به دلیل ماهیت off-policy ممکن است کمی بهتر از SARSA عمل کند.

- n = 1 سریعترین همگرایی دارد (با توجه به شیب و نه مقدار های regret) ، زیرا فقط بر پاداشهای فوری و حالت بعدی تکیه دارد.

با این حال، حسرت کلی n=1 از دو حالت دیگر بیشتر است.



- n = 3 بهترین تعادل بین سرعت همگرایی و حسرت کلی را دارد و از هر دو مقدار عمکری بهتری دارد.
- n = 5 پاداشهای بلندمدت بیشتری را در نظر میگیرد تا بتواند به سیاستهای نهایی بهتری برسد، اما سرعت همگرایی پایینتری دارد.

ج)

برای اپسیلون ثابت باتوجه به اینکه میتوانست در loop باقی بماند و به خانه نهایی نرسد، یک ورژن جدید از الگوریتم Monte Carlo Off-Policy نوشتم، با این تفاوت تا حداکثر ۱۰۰۰۰ بار حلقه اجرا شود، و سپس درصورت نرسیدن به خانه هدف امتیاز ۱۰۰۰- بگیرد.
درواقع برای اپسیلون ثابت هیچ تضمینی به همگرایی وجود ندارد و سرعت همگرایی و مقدار همگرایی را نمیتوان تعریف کرد.

در اخر میبینیم حتی با وجود این تغییر، الگوریتم با کاهش تدریجی اپسیلون نسبت به اپسیلون ثابت عملکرد بهتری دارد، (با همان نسخه اولیه به جواب بهینه همگراه میشود)، در این حالت در ابتدا عامل برای کاوش شانس بیشتری دارد و به سرعت عمل بهینه را شناسایی میکند. این امر باعث همگرایی سریعتر و حسرت کمتر میشود (حسرت رسم نشده اما میتوان به صورت متقارن پاداش در نظرگرفته شود)، در حالتی که ا اپسیلون ثابت است به دلیل عدم تنظیم میزان کاوش به درستی، همگرایی کندتری دارد (در ورژن دوم و در ورژن اصلی Monte Carlo Off-Policy اصلا همگرایی ندارد) و حسرت بیشتری نشان میدهد.

پس استفاده از نسخه دوم الگوریتم MC تضمین میکند که عامل از لحاظ کاوش و بهره برداری تعادل بهتری پیدا کند و به حالت نهایی برسد.

```
total_rewards_off_policy_mc = run_off_policy_monte_carlo_experiment(
    env, num_runs, num_episodes, learning_rate, discount, epsilon = 0.1)

plot_result(total_rewards_off_policy_mc, [], num_episodes, num_runs, figsize=(15, 6),
            title="Off-Policy Monte Carlo")
```

⊗ 11m 8.9s

در ورژن اولی میبنیم اجرا ۱۱ دقیقه طول کشیده ولی به جوابی نرسیدیم که علت آن عدم تضمین همگرایی الگوریتم در این حالت است.

منابع:

سوال اول:

https://www.geeksforgeeks.org/sparse-rewards-in-reinforcement-learning/

https://careerfoundry.com/en/blog/data-analytics/human-in-the-loop/

سوال دوم:

an Introduction Learning Reinforcement -> فصل ششم

Hands_On3

سوال سوم:

https://www.geeksforgeeks.org/on-policy-vs-off-policy-methods-reinforcement-learning/

Off-policy vs. On-policy Reinforcement Learning | Baeldung on Computer Science

سوال چهارم:

Hands_On3

*** Promptهای LLM تمامی سوالات پیوست شده است. (روز اخر کار نکرد، نهایتا از لینکهای بالا برای سوالات استفاده شده است.)